



Padrões de Projeto (MVC e MVP)

Desenvolvimento Web Back-end

Objetivos da Aula

- compreender o conceito de padrão de projeto;
- identificar o problema da ausência de organização;
- compreender o padrão MVC;
- compreender o padrão MVP;
- identificar as responsabilidades de cada componente;
- acompanhar o fluxo de uma solicitação em MVC e MVP;
- comparar MVC e MVP;
- reconhecer aplicações práticas desses padrões.

Onde estamos na disciplina



- Na aula anterior aprendemos a distribuir responsabilidades entre componentes e camadas.
- Agora vamos estudar padrões que ajudam a organizar essas responsabilidades dentro de uma aplicação.

O que acontece quando colocamos toda a aplicação em um único lugar?



Um Sistema pode ficar difícil de manter



- Problemas:
 - código difícil de entender;
 - alterações podem afetar várias partes;
 - dificuldade de testar;
 - responsabilidades misturadas;
 - crescimento desorganizado.



Precisamos organizar as responsabilidades



Quando cada camada tem uma responsabilidade clara e bem separa, o sistema se torna organizado, flexível e sustentável

O que são Padrões de Projeto?



- De forma ampla, um padrão de projeto é uma **solução reutilizável e consagrada para problemas recorrentes** no desenvolvimento de software.
- Funcionam como plantas arquiteturais ou guias consolidados para estruturar o código de maneira limpa, modular e extensível.

Padrão de projeto não é uma biblioteca nem um código pronto.



Abrangência dos Padrões de Projeto



Os padrões costumam ser divididos em níveis de abrangência:

1. Padrões Criacionais
2. Padrões Estruturais
3. Padrões Comportamentais
4. Padrões Arquiteturais (ou de Nível Macro)



PUC MINAS

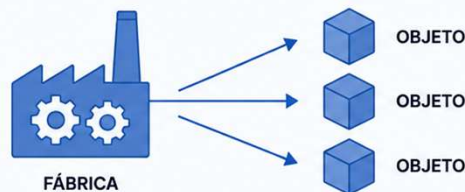
Padrões Criacionais



1. PADRÕES CRIACIONAIS

(Como os objetos são criados)

Lidam com a forma de instanciar objetos.



Exemplos: Singleton, Factory Method



PUC MINAS

Lidam com a forma como os objetos são instanciados

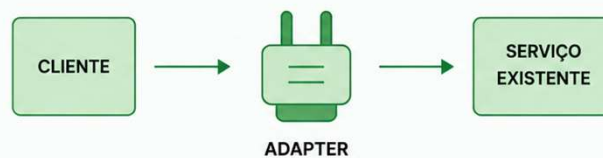
Padrões Estruturais



2. PADRÕES ESTRUTURAIS

(Como os objetos são estruturados)

Organizam classes e objetos em estruturas maiores.



Exemplos: Adapter, Facade

Organizam classes e objetos em estruturas maiores para garantir flexibilidade.

PUC MINAS

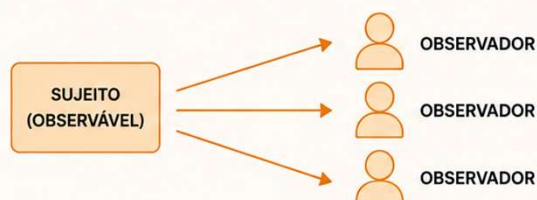
Padrões Comportamentais



3. PADRÕES COMPORTAMENTAIS

(Como os objetos interagem)

Gerenciam a comunicação e a divisão de responsabilidades.



Exemplos: Observer, Strategy

Gerenciam a comunicação e a divisão de responsabilidades entre os objetos.

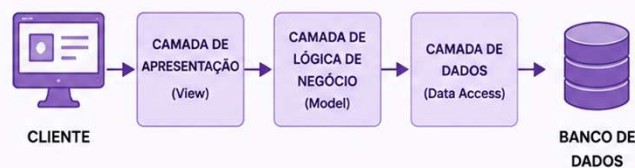
PUC MINAS

Padrões Arquiteturais (de Nível Macro)

4. PADRÕES ARQUITETURAIS

(Estrutura geral da aplicação)

Definem a arquitetura da aplicação e a organização em camadas.



Exemplos: MVC, MVP

Definem a estrutura geral de uma aplicação inteira, organizando seus subsistemas e camadas.

PUC MINAS

Organizando o Front-end e o Back-end: MVC e MVP

Para aplicações que lidam com interfaces de usuário, um desafio arquitetural é separar a lógica de negócios da apresentação visual.
(princípio da *Separação de Preocupações* ou *Separation of Concerns*).

É nesse contexto que se destacam as abordagens mais importantes da engenharia de software:

MVC e MVP

PUC MINAS

MVC (Model-View-Controller)



Visão Geral

- Model: Gerencia os dados e a lógica de negócios pura.
- Responsável principal:
 - Mantém os dados da aplicação;
 - Estado da aplicação;
 - Regras relacionadas ao domínio.



MVC (Model-View-Controller)



Visão Geral

- **View:** Cuida da interface visual e da exibição dos dados para o usuário.
- Exemplos:
 - páginas;
 - telas;
 - formulários;
 - componentes visuais.



MVC (Model-View-Controller)

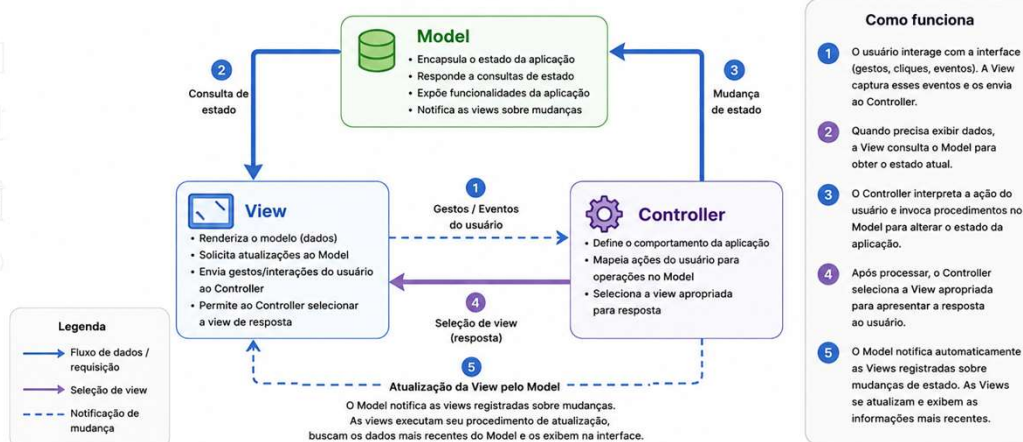


Visão Geral

- **Controller:** Intermedia as ações do usuário, interpretando comandos e atualizando o Model ou a View.
- É amplamente utilizado no desenvolvimento web (tanto do lado do servidor em frameworks como Spring/ASP.NET quanto no cliente).
- Responsável por receber e coordenar as solicitações.
- Pode:
 - receber uma ação;
 - interpretar a solicitação;
 - acionar o Model;
 - selecionar uma View;
 - coordenar o fluxo.

PUC MINAS

Fluxo básico do MVC

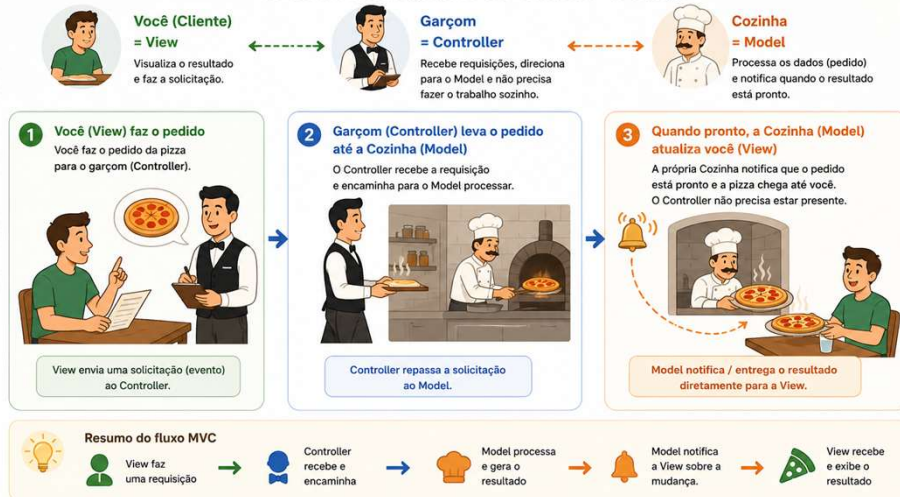


PUC MINAS

MVC - Metáfora do Restaurante



No cenário MVC: o garçom é um "Controller" de tráfego



PUC MINAS

MVP (Model-View-Presenter)



Visão Geral

- Uma evolução voltada a resolver limitações de acoplamento, muito comum em aplicações desktop e mobile.
- Model:
 - Mantém os dados da aplicação;
 - Estado da aplicação;
 - Regras de negócio.

PUC MINAS

MVP (Model-View-Presenter)



Visão Geral

- **View:**
 - Uma interface extremamente passiva (Passive View);
 - Responsável apenas por desenhar elementos e repassar eventos do usuário.
- **Exemplos:**
 - páginas;
 - telas;
 - formulários;
 - componentes visuais.

MVP (Model-View-Presenter)



Visão Geral

- Uma evolução voltada a resolver limitações de acoplamento, muito comum em aplicações desktop e mobile.
- **Presenter:**
 - O "cérebro" intermediário que possui referência direta à View e ao Model, supervisionando toda a sincronização e lógica de apresentação de forma explícita;
 - Recebe eventos da View;
 - Solicita informações ao Model;
 - Processa resultados;
 - Determina o que deve ser apresentado.

Fluxo básico do MVP

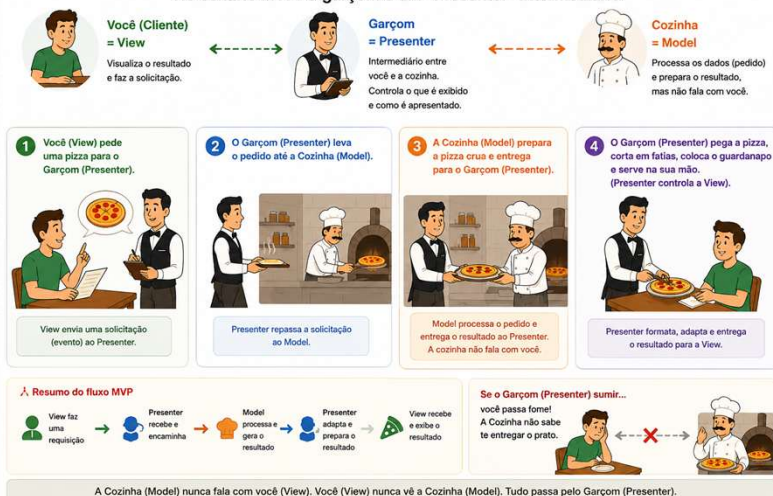


PUC MINAS

MVP - Metáfora do Restaurante



No cenário MVP: O garçom é um "Presenter" intermediário



PUC MINAS

Separação de Responsabilidades



MVC

Controller coordena

View Apresenta

Model contém dados/regras

MVP

Presenter coordena

View Apresenta

Model contém dados/regras



Principal Diferença



- **No MVC:**
 - O Controller participa do fluxo da aplicação e coordena Model e View.
- **No MVP:**
 - O Presenter assume um papel central na lógica relacionada à apresentação.



Por que estudar esses padrões?



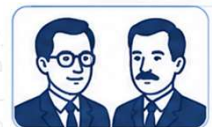
- Compreender a diferença entre os padrões arquiteturais MVC e MVP evita que sua aplicação vire um código monolítico e confuso.
- Explorar como aplicar esses conceitos e garantir que aplicações web possuam alta coesão, facilidade para testes unitários e manutenibilidade a longo prazo.



Arquitetura MVC (Model-View-Controller)



- O **Model-View-Controller (MVC)** foi introduzido originalmente por G. E. Krasner e S. T. Pope no Smalltalk'80 como um padrão de design pioneiro.



G. E. Krasner S. T. Pope

- Objetivo central: **separação de preocupações (Separation of Concerns)**.
 - Divide as responsabilidades do sistema em três camadas distintas de forma que o domínio da aplicação permaneça completamente independente da interface gráfica.



O Componente Modelo do MVC

•Responsabilidade:

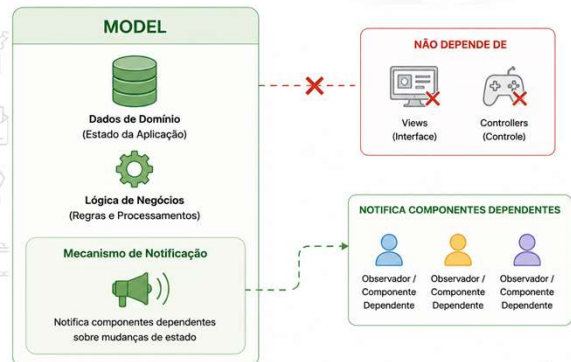
- Encapsula o estado da aplicação, os dados de domínio e a lógica de negócios.

•Independência:

- O Model funciona de maneira autônoma, sem conhecimento direto da interface do usuário (Views ou Controllers).

•Mecanismo de Notificação:

- Expõe procedimentos e mantém um registro de componentes dependentes para propagar alterações de estado (via observadores ou eventos).



O Model é o coração da aplicação: gerencia dados e regras de negócio, operando de forma independente e comunicando mudanças de estado aos interessados.

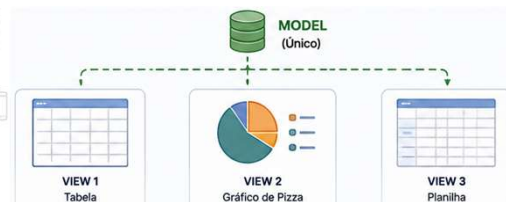
O Componente Visão do MVC

•**Responsabilidade:** Responsável pela representação visual e exibição dos dados aos usuários.

- Ex.: telas, componentes HTML, elementos gráficos.

•**Interação:** Exibe o conteúdo obtido a partir do Model e envia gestos/ações do usuário para o controlador.

• **Múltiplas Visões:** Um único Model pode possuir várias Views simultâneas.



A View se preocupa apenas com como os dados são apresentados ao usuário.

O Componente Controlador do MVC

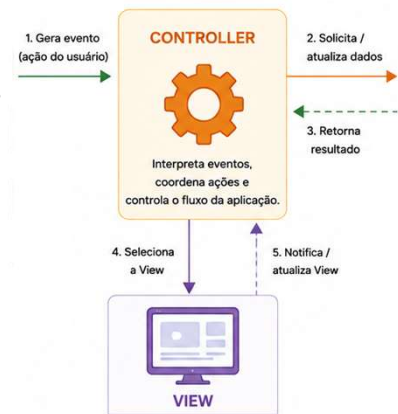


- Responsabilidade:** Define o comportamento da aplicação e gerencia o fluxo de controle.

- Interação com o Usuário:** É o ponto de entrada para as ações do usuário.

- Ex.: cliques de mouse, eventos de teclado, requisições HTTP.

- Mapeamento:** Converte eventos em solicitações de serviço para o Model e seleciona a View de resposta apropriada.



Mecanismos de Sincronização



- A consistência entre o estado do domínio e a interface gráfica é mantida através de duas abordagens principais:

1. **Sincronização por Fluxo (Flow Synchronization).**

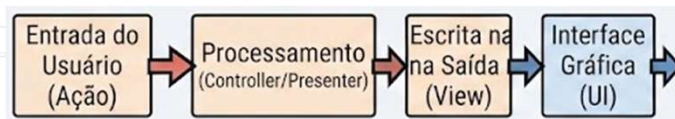
2. **Sincronização por Observador (Observer Synchronization)**

Sincronização por Fluxo



- Baseada em chamadas sequenciais diretas de comandos leitura de entrada --> processamento --> escrita na saída.

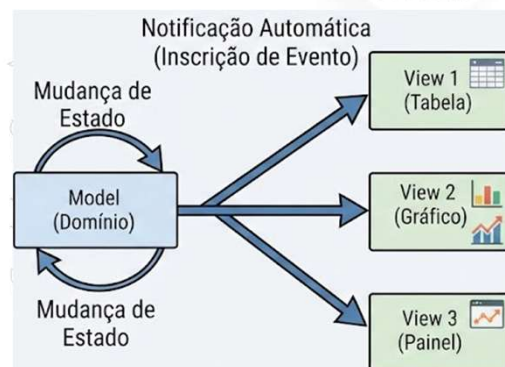
- Comum em aplicações web tradicionais e abordagens de visualização simples.



Sincronização por Observador



- Baseada em inscrições de eventos.
- O Model notifica automaticamente todas as Views inscritas sempre que seus dados sofrem alterações.
 - Permite atualizações imediatas em múltiplos formatos de exibição.



Arquitetura MVP (Model-View-Presenter)



- A família de padrões **MVP (Model-View-Presenter)** evoluiu tendo o MVC como ponto de partida, mas adaptando papéis e regras de cooperação para alcançar maior flexibilidade.
- Introduzida originalmente pela IBM/Taligent e popularizada pelo Dolphin Smalltalk.



Componente Modelo do MVP



•Model (Modelo):

- Representa os dados e a lógica de domínio.
- Dependendo da variação do padrão (como no *Supervising Presenter*), ele também pode conter dados necessários para renderizar corretamente o estado da tela.
- O Model mantém-se independente da interface gráfica, mas pode fornecer mecanismos de notificação de alteração de dados.

○ OBS: no padrão de projeto *Supervising Presenter*, os eventos da interface são capturas e a atualização da View ocorre diretamente quando necessário.



Componente Visão do MVP



- **View (Visão):**

- Responsável por exibir os dados na interface do usuário e lidar com interações básicas (como capturar cliques e repassá-las).
- Em muitas variações, a View possui pouca ou nenhuma lógica interna, atuando por meio de interfaces (IView) para garantir maior testabilidade.



Componente Apresentador do MVP



- **Presenter (Apresentador):**

- O coração da aplicação no padrão MVP.
- O Presenter gerencia a sincronização entre a View e o Model, manipula a entrada do usuário tratada pela View, executa regras de negócio/validação e atualiza diretamente a View quando necessário.



Principais Variações da Família MVP



•Dolphin Smalltalk MVP:

- O núcleo do comportamento da aplicação fica no **Presenter**.
- O **Presenter** possui acesso direto à **View** para supervisioná-la e modificá-la diretamente quando necessário (por exemplo, alterando propriedades visuais complexas).
- Suporta **data binding** (vinculação dos dados) bidirecional e delegação de eventos.



Principais Variações da Família MVP



•Supervising Presenter:

- O **Model** pode conter dados de estado da visão além dos dados de domínio puro.
- A **View** reflete os dados do **Model** através de **data binding** simples de mão única, enquanto o **Presenter** intervém diretamente apenas em casos complexos de lógica de interface.
- A **View** não possui referência direta ao **Presenter**, o que reduz o acoplamento.



Principais Variações da Família MVP



•Passive View:

- Utiliza sincronização por fluxo (*flow synchronization*) em vez de observadores.
- A *View* é mantida o mais simples possível (possuindo apenas getters, setters e repasse de eventos), sem nenhuma lógica real, o que torna os testes unitários de interface extremamente triviais.
- Toda a sincronização e manipulação de estado residem explicitamente no *Presenter*.



Vantagens e Limitações do MVP



•Vantagens:

- Flexibilidade:** Permite ajustar a alocação de responsabilidades a uma ampla gama de cenários de aplicação.
- Testabilidade:** Especialmente na variante *Passive View*, a separação rigorosa facilita a criação de testes automatizados para a lógica de apresentação.
- Desacoplamento:** O uso de interfaces na *View* permite que o *Presenter* interaja com diferentes implementações visuais.

•Limitações:

- Menos Restrito:** Por ser flexível, o MVP pode ser menos rígido quanto à separação estrita de camadas, o que pode aumentar a complexidade do código e dificultar a manutenção se não for bem aplicado.



Principais Diferenças



Padrão	Vantagens Principais	Limitações e Desvantagens
MVC	Separação clássica de conceitos. Excelente para aplicações Web tradicional - separação natural entre cliente e servidor.	Dificuldade no gerenciamento de estados puramente visuais. Assume componentes desacoplados que muitas vezes vêm fundidos em widgets modernos.
MVP	Extrema flexibilidade na alocação de responsabilidades. Adaptável a diversos cenários de aplicação e testes unitários.	Menos restrito quanto à separação estrita de camadas, o que pode aumentar a complexidade do código se mal aplicado.

O que aprendemos?



- O aluno deve conseguir responder:
 - O que é um padrão de projeto?
 - O que significa MVC?
 - Qual o papel do Model?
 - Qual o papel da View?
 - Qual o papel do Controller?
 - O que é MVP?
 - Qual o papel do Presenter?
 - Qual a principal diferença entre MVC e MVP?



Na aula de laboratório iremos
abordar de forma prática os
conceitos vistos até aqui. Até lá!



Referências



Syromiatnikov, A.; Weyns, D. *A Journey Through the Land of Model-View- Designs Patern**. Linnaeus University.

Pethuru, A.; Subramanian, R. H.; Chelliah, J. P. *Architectural Patterns*. Packt Publishing.

Krasner, G. E.; Pope, S. T. *A cookbook for using the model-view controller user interface paradigm in smalltalk-80*.





PUC Minas

© PUC Minas • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.
Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.